

Welcome to

Web Attacks I – Attacks

Saint Cyber – Cyber Akademiet, 2026

Henrik Kramselund he/him han/ham hlk@zencurity.com @kramse

Slides are available as PDF, https://codeberg.org/kramse
academy-2026-day2-web-attacks-I.tex in the repo security-courses

© license CC BY 3.0. 2026 Henrik Kramselund

Goals for today



- Identify and explain common web-application weaknesses
- Cover SQL injection, cross-site scripting and cross-site request forgery
- Cover session handling, file-inclusion flaws and security misconfiguration
- Relate them to the OWASP Top 10

Plan for today



- Why web hacking – and the OWASP Top 10 as a map
- Injection: SQL injection
- Client-side: XSS and CSRF
- Sessions, file inclusion and misconfiguration

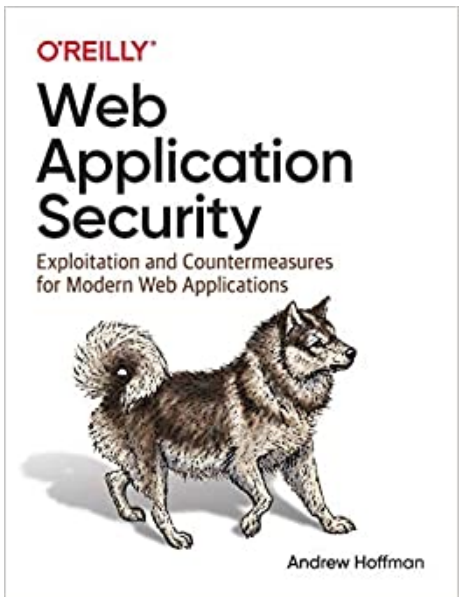
Why talk about hacking web



- Everything is web today
- HTTPS/TLS used for data transport in most applications
- Web APIs are used across the industry
- Even *desktop* and *mobile* application are rarely more than a specialized browser like electronjs.org – Chromium and Node.js
- Also software security and web security has a HUGE overlap

Photo by Pawel Janiak on Unsplash

Web Application Security



Web Application Security, Andrew Hoffman, 2020, ISBN: 9781492053118 called WAS

OWASP Top Ten





> The OWASP Top Ten provides a minimum standard for web application security. The OWASP Top Ten represents a broad consensus about what the most critical web application security flaws are.

The Open Web Application Security Project (OWASP)

OWASP produces lists of the most common types of errors in web applications

http://www.owasp.org

Create Secure Software Development Lifecycle

SQL injection



SQL Injection FAQ from Chip Andrews 2001 http://www.sqlsecurity.com:

```
Set myRecordset = myConnection.execute
("SELECT * FROM myTable
WHERE someText ='" & request.form("inputdata") & "'")
```

Using input `exec master..xp_cmdshell 'net user test testpass /ADD' --`

Will perform

```
SELECT * FROM myTable
WHERE someText ='' exec master..xp_cmdshell 'net user test testpass /ADD'--
```

Depending on the access rights this might have full authorization in the database!

Cross-site scripting



When logged into a site, we have a session identifier – being presented with each HTTP request

If an attacker can *activate* javascript that access and re-send this to another site we have a cross-site scripting attack

We often check using some parameter with code, like this:

```
<A HREF="http://example.com/comment.cgi?
mycomment=<SCRIPT>malicious code</SCRIPT>
">Click here</A>
```

If this code is returned, and active, as part of the returned page and HTML we have a cross-site scripting possibility

This code is executed in the user browser, with the same permissions as the user

11. Cross-Site Request Forgery (CSRF)



Sometimes we already know an API endpoint exists that would allow us to perform an operation we wish to perform, but we do not have access to that endpoint because it requires privileged access (e.g., an admin account).

In this chapter, we will be building Cross-Site Request Forgery (CSRF) exploits that result in an admin or privileged account performing an operation on our behalf rather than using a JavaScript code snippet.

CSRF attacks take advantage of the way browsers operate and the trust relationship between a website and the browser. By finding API calls that rely on this relationship to ensure security—but yield too much trust to the browser—we can craft links and forms that with a little bit of effort can cause a user to make requests on his or her own behalf—unknown to the user generating the request.

- Often seen in small CPE routers in homes, if the user activates and evil link, their router might be reconfigured or taken over

Session IDs



- Session IDs tie the user with the state on the server
- Must be randomly assigned, otherwise an attacker can guess a valid ID
- Common problems, time based or predictable in some way
- Check code for generating IDs or measure - Phase Space Analysis

File inclusion (LFI / RFI)



- **FIXME:** not in pentest-II – add local/remote file inclusion (subject plan obj 7)

Security misconfiguration



- **FIXME:** add defaults / verbose errors / missing headers (subject plan obj 7)

Summary



- The main web weaknesses: injection, XSS, CSRF, sessions, file inclusion, misconfiguration
- Next: the tools, labs and defences